



PRÁTICAS DE SISTEMAS DIGITAIS - T02

Relatório de Práticas em Sistemas Digitais Segunda Unidade

{joaoccs,vonvictor,177013ot}@academico.ufs.br

João Carlos Cruz Santos, João Victor Carvalho Simões, Otávio
Augusto Batista Santos Filho

Orientador: Calebe Micael de Oliveira Conceição

Departamento de Computação – Universidade Federal de Sergipe (UFS) São
Cristóvão, SE – Brasil

Relatório da prática 06 – Lab 6

São Cristóvão, Sergipe 2025

SUMÁRIO

0. Resumo

1. Introdução

2. Atividade 1

2.1 Entendendo o Circuito

2.2 Pergunta a

2.3 Pergunta b

2.4 Pergunta c

3. Atividade 2

3.1 Pergunta a

3.2 Pergunta b

3.3 Pergunta c

4. Hora- Trabalho/ Questões Individuais

4.1 Pergunta 1

4.2 Pergunta 2

5. Resultados e discussão

6. Conclusão

7. Referências Bibliográficas

0. Resumo

Este laboratório tem como objetivo exercitar técnicas de minimização lógica de funções booleanas, conforme proposto no Laboratório 06 da disciplina Práticas em Sistemas Digitais. São abordados métodos manuais de simplificação, por meio do uso de Mapas de Karnaugh, bem como métodos algorítmicos clássicos, com destaque para o algoritmo de Quine-McCluskey e o uso de ferramentas CAD/EDA, como o Espresso, utilizando funções descritas no formato PLA. O trabalho busca consolidar a compreensão teórica e prática do processo de minimização lógica e sua importância no projeto de circuitos digitais eficientes.

1. Introdução

O presente relatório tem como propósito descrever e analisar as atividades desenvolvidas no Laboratório 06 da disciplina Práticas em Sistemas Digitais, cujo foco principal é a minimização lógica de funções booleanas. A partir de uma tabela-verdade fornecida na súmula do laboratório, são aplicados métodos formais de simplificação, inicialmente por meio do Mapa de Veitch-Karnaugh e, posteriormente, por meio do algoritmo de Quine-McCluskey. Além disso, são exploradas ferramentas CAD/EDA para automatização do processo de minimização, permitindo a comparação entre os resultados obtidos manualmente e aqueles gerados por ferramentas computacionais. Dessa forma, busca-se aprofundar o entendimento sobre otimização lógica e sua relevância no projeto de sistemas digitais.

2. Atividades

2.1 Entendendo o Circuito

Tabela 1. Tabela-verdade de uma Função F qualquer

A	B	C	D	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Circuito usado como referencia

Atividade 01 – Minimização com Mapa de Karnaugh (K-maps)

2.2) *Expresse a saída F como uma função Soma de Produtos (Sum Of Products – SOP).*

A	B	C	D	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1

0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Conforme pode-se observar na tabela-verdade acima, a saída F tem valor 1 em três casos: a) 0000; b) 0100; c) 1000. Na forma de expressão lógica, mais especificamente na forma canônica **de soma de produtos**, a tabela é representada assim $F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D}$.

2.3) *Usando K-maps, encontre a forma minimizada da função SOP de F encontrada.*

Existe a possibilidade de minimizarmos a expressão anterior. Um método bastante utilizado para isso é o **Mapa de Veitch-Karnaugh (K-map)**. Essa ferramenta organiza as combinações das variáveis de forma bidimensional, representando cada combinação em um quadrado. Cada quadrado corresponde a uma linha da tabela-

verdade, facilitando a visualização e a simplificação da função lógica.

Como, no exemplo dado, há quatro variáveis, inicialmente elas serão divididas em dois agrupamentos de duas variáveis cada: **AB** e **CD**. Cada agrupamento terá quatro combinações:

Para AB: AB, $\bar{A}\bar{B}$, $\bar{A}B$ e $A\bar{B}$;

Para CD: CD, $\bar{C}\bar{D}$, $C\bar{D}$ e $\bar{C}D$.

O primeiro agrupamento formará as **linhas** do mapa, enquanto o segundo formará as **colunas**, gerando $4 \times 4 = 16$ quadrados, correspondentes às 16 linhas da tabela-verdade.

É importante destacar que, ao organizar essas variações, cada combinação adjacente deve diferir **apenas por um bit**. Isso garante que o mapa de Karnaugh preserve a propriedade necessária para a simplificação das funções lógicas.

Dessa forma, abaixo, segue o mapa de **Veitch-Karnaugh, para o exemplo dado:**

	$\bar{C}\bar{D}$	$C\bar{D}$	CD	$\bar{C}D$
$\bar{A}\bar{B}$	1	0	0	0
$\bar{A}B$	1	0	0	0
$A\bar{B}$	0	0	0	0
AB	1	0	0	0

O passo seguinte é agrupar os valores 1 subjacentes, mantendo apenas as variáveis cujo valor não muda. As variáveis cujo valor muda são descartadas.

	$\bar{C}\bar{D}$	$C\bar{D}$	CD	$\bar{C}D$
--	------------------	------------	------	------------

$\bar{A}\bar{B}$	1	0	0	0
$\bar{A}B$	1	0	0	0
$A\bar{B}$	0	0	0	0
AB	1	0	0	0

No agrupamento acima, estão as combinações $\bar{A}\bar{B}\bar{C}\bar{D}$ e $\bar{A}\bar{B}C\bar{D}$. Apenas $\bar{B}\bar{C}\bar{D}$ não mudam. Ou seja, para qualquer valor de A, se $\bar{B}\bar{C}\bar{D}$, então a saída é 1. Logo, A é uma variável não necessária nessas combinações. e pode ser excluída.

	$\bar{C}\bar{D}$	$C\bar{D}$	CD	$\bar{C}D$
$\bar{A}\bar{B}$	1	0	0	0
$\bar{A}B$	1	0	0	0
$A\bar{B}$	0	0	0	0
AB	1	0	0	0

No agrupamento acima, estão as combinações $\bar{A}\bar{B}\bar{C}\bar{D}$ e $\bar{A}\bar{B}C\bar{D}$. Apenas $\bar{A}\bar{C}\bar{D}$ não mudam. Ou seja, para qualquer valor de B, se $\bar{A}\bar{C}\bar{D}$, então a saída é 1. Logo, B é uma variável não necessária nessas combinações. e pode ser excluída.

Com todo esse processo, $F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}D$ foi minimizada para $F = \bar{A}\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D}$, ou $F = \bar{C}\bar{D}(\bar{A} + \bar{B})$.

2.4) Construa a rede de transistores que implementa a função lógica minimizada encontrada.

A função lógica minimizada considerada neste projeto é dada por:

$$F = \bar{C} \bar{D} (\bar{A} + \bar{B})$$

Para a implementação dessa função em tecnologia CMOS, utiliza-se a abordagem clássica baseada na separação entre a rede de pull-down (NMOS) e a rede de pull-up (PMOS), as quais são conectadas a um único nó de saída. Essa metodologia garante correto funcionamento lógico e baixo consumo de potência estática.

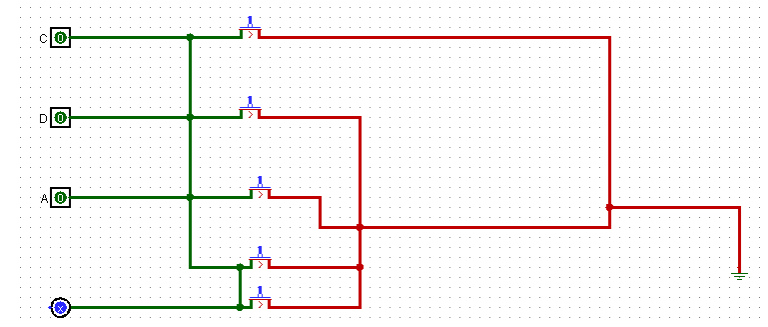
Inicialmente, constrói-se a rede de pull-down, responsável por conduzir a saída ao nível lógico baixo quando a função for falsa. Para isso, parte-se da expressão complementar da função lógica:

$$\bar{F} = C + D + AB$$

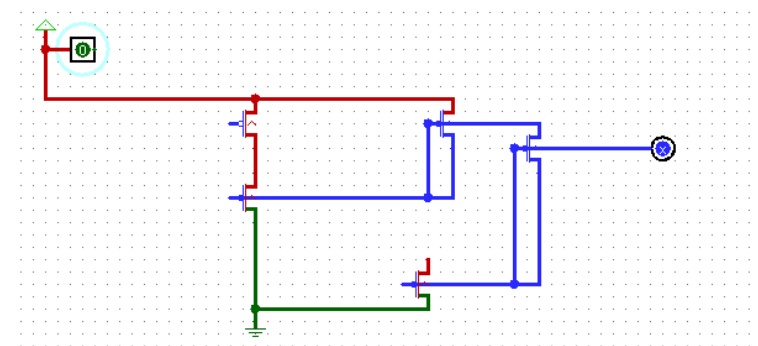
Essa expressão indica que a saída deve ser conectada ao terra sempre que $C = 1$, $D = 1$ ou quando $A = 1$ e $B = 1$ simultaneamente. Em termos de implementação em nível de transistores, a operação lógica OU é representada por transistores em paralelo, enquanto a operação lógica E é representada por transistores em série.

Assim, a rede NMOS é composta por três ramos em paralelo: um transistor NMOS controlado pela variável C, um transistor NMOS controlado pela variável D e um terceiro ramo formado por dois transistores NMOS conectados em série, controlados pelas variáveis A e B.

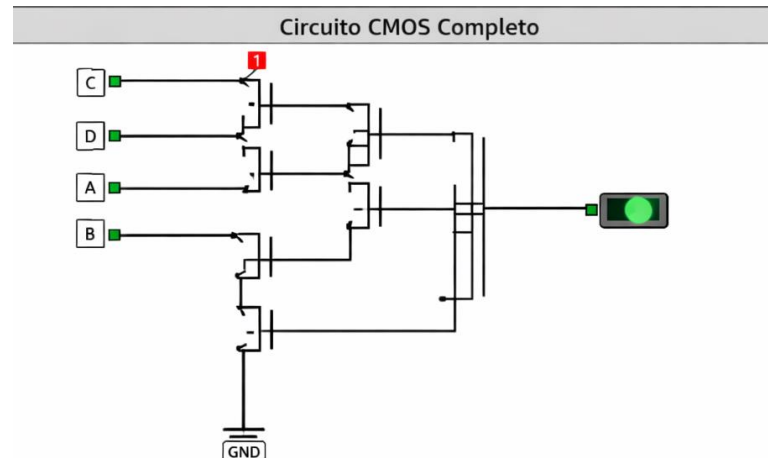
Rede NMOS (Pull-Down)



Rede PMOS (Pull-Up)



Circuito CMOS Completo



A interligação das redes de pull-up e pull-down resulta no circuito CMOS completo. Ambas as redes compartilham o mesmo nó de saída, sendo a rede PMOS conectada ao VDD e a rede NMOS conectada ao GND. Nunca ocorre condução direta entre VDD e GND, garantindo consumo estático praticamente

nulo.

3. Atividade 02 – Implementação do Algoritmo de Quine-McCluskey. Com base nos Slides de Aula, na Bibliografia de Referência desta aula, nos benchmarks indicados:

Com base nos slides de aula, na bibliografia de referência e nos benchmarks indicados, esta atividade analisa o processo de minimização lógica por meio de ferramentas CAD e da implementação completa do algoritmo de Quine-McCluskey.

3.1)Execute a ferramenta indicada pelo docente no quadro para experimentar o processo de minimização usando ferramentas CAD, observando as entradas e saídas geradas.

Utilizou-se a ferramenta Espresso para realizar a minimização automática de funções descritas em formato PLA, observando-se entradas e saídas geradas.

3.2) Crie um programa similar que implementa o algoritmo de Quine-McCluskey para minimização de funções lógicas. Implemente o suporte à entrada em formato PLA.

Foi desenvolvido um programa completo em linguagem C que implementa o algoritmo de Quine-McCluskey, incluindo leitura do formato PLA, geração de implicantes primos, tabela de cobertura e seleção de implicantes essenciais.

Apêndice – Implementação do Algoritmo de Quine-McCluskey em C

A seguir apresenta-se uma implementação completa do algoritmo de Quine-McCluskey, com suporte à leitura de funções no formato PLA, geração de implicantes primos e identificação de termos essenciais.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define MAX_VARS 32
#define MAX_TERMS 2048

typedef struct {
    char bits[MAX_VARS + 1]; // String binária (ex: "01-1")
    bool used;                // Flag para marcar se foi agrupado
    bool is_minterm;          // Se é um mintermo original
    int *covered_minterms;    // IDs dos mintermos cobertos (para etapa de chart)
    int count_covered;
} Term;

typedef struct {
    Term *terms;
    int count;
} Group;

int num_inputs = 0;
int num_outputs = 0;

// --- Funções Auxiliares ---

// Conta quantos bits diferem entre duas strings
int diff_bits(const char *a, const char *b, int len, int *idx) {
    int diff = 0;
    for (int i = 0; i < len; i++) {
        if (a[i] != b[i]) {
            diff++;
            *idx = i;
        }
    }
    return diff;
}
```

```

}

// Cria um novo termo combinando dois
// (coloca '-' na diferença)
Term merge_terms(Term a, Term b, int
diff_idx) {
    Term res;
    strcpy(res.bits, a.bits);
    res.bits[diff_idx] = '-';
    res.used = false;
    res.is_minterm = false;

    // Na implementação completa de
    cobertura, aqui uniríamos as listas de
    mintermos cobertos
    // Para simplificação visual, não
    vamos gerenciar arrays dinâmicos
    complexos aqui
    return res;
}

// Verifica se um termo já existe no
grupo (evitar duplicatas)
bool term_exists(Group *g, const char
*bits) {
    for (int i = 0; i < g->count; i++)
    {
        if (strcmp(g->terms[i].bits,
bits) == 0) return true;
    }
    return false;
}

// Adiciona termo ao grupo
void add_term(Group *g, Term t) {
    if (term_exists(g, t.bits)) return;
    if (g->count >= MAX_TERMS) {
        printf("Erro: Limite de termos
atingido.\n");
        exit(1);
    }
    g->terms[g->count++] = t;
}

```

```

// Expande entradas PLA (ex: 1-0 -> 100,
110) recursivamente
void expand_pla_input(char *cube, Group
*minterms_list) {
    char *dash = strchr(cube, '-');
    if (dash == NULL) {
        // Base: sem don't cares, é um
        mintermo
        Term t;
        strcpy(t.bits, cube);
        t.used = false;
        t.is_minterm = true;
        add_term(minterms_list, t);
        return;
    }

    // Recursão: substitui '-' por '0'
    e depois por '1'
    int index = dash - cube;

    cube[index] = '0';
        expand_pla_input(cube,
minterms_list);

    cube[index] = '1';
        expand_pla_input(cube,
minterms_list);

    cube[index] = '-'; // Restaura para
voltar na recursão
}

// Leitura de Arquivo PLA Simples (.i,
.o, .e)
void read_pla(const char *filename,
Group *minterms) {
    FILE *f = fopen(filename, "r");
    if (!f) { perror("Erro ao abrir
arquivo"); exit(1); }

    char line[256];
    while (fgets(line, sizeof(line),
f)) {

```

```

        if (line[0] == '#') continue;
// Comentário
        if (line[0] == '.') {
            if (strncmp(line, ".i", 2)
== 0) sscanf(line, ".i %d",
&num_inputs);
            if (strncmp(line, ".o", 2)
== 0) sscanf(line, ".o %d",
&num_outputs);
            if (strncmp(line, ".e", 2)
== 0) break;
            continue;
        }

// Leitura dos dados: Entrada
Saída
char input_part[MAX_VARS + 1];
char output_part[10];

// Formato simples assumido:
"BITS SAIDA"
        if (sscanf(line, "%s %s",
input_part, output_part) == 2) {
            // Se a saída for 1,
            consideramos para minimização (SOP)
            // Se o PLA definir output
            '1', processamos.
            // Nota: Este simulador foca
            em funções de 1 saída ou considera a
            primeira.
            if (strchr(output_part,
            '1')) {
                expand_pla_input(input
            _part, minterms);
            }
        }
        fclose(f);
    }

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s
<arquivo.pla>\n", argv[0]);

```

```

        return 1;
    }

    Group current_group = { .terms =
malloc(MAX_TERMS * sizeof(Term)),
.count = 0 };
    Group next_group = { .terms =
malloc(MAX_TERMS * sizeof(Term)),
.count = 0 };
    Group prime_implicants = { .terms =
malloc(MAX_TERMS * sizeof(Term)),
.count = 0 };

    printf("--- Lendo Arquivo PLA ---
\n");
    read_pla(argv[1], &current_group);
    printf("Mintermos expandidos:
%d\n", current_group.count);
    printf("Inputs: %d\n\n",
num_inputs);

    int pass = 1;
    bool changes = true;

    // --- Passo 1: Encontrar
    Implicantes Primos ---
    while (changes) {
        changes = false;
        next_group.count = 0; // Limpa
        o próximo grupo

        // Zera flags de usado para o
        grupo atual (novo passo)
        for (int i = 0; i <
current_group.count; i++)
            current_group.terms[i].used = false;

        // Compara todos com todos
        for (int i = 0; i <
current_group.count; i++) {
            for (int j = i + 1; j <
current_group.count; j++) {
                int diff_idx = -1;

```

```

                                if
(diff_bits(current_group.terms[i].bits
,          current_group.terms[j].bits,
num_inputs, &diff_idx) == 1) {
                                // Se diferem em
apenas 1 bit, combina
                                Term merged =
merge_terms(current_group.terms[i],
current_group.terms[j], diff_idx);
                                add_term(&next_group,
merged);

                                current_group.term
s[i].used = true;
                                current_group.term
s[j].used = true;
                                changes = true;
                                }
                                }
                                }

                                // Os termos que NÃO foram
combinados nesta rodada são Primos
                                for (int i = 0; i <
current_group.count; i++) {
                                if
(!current_group.terms[i].used) {
                                add_term(&prime_implic
ants, current_group.terms[i]);
                                }
                                }

                                if (changes) {
                                // Copia next_group para
current_group para a próxima iteração
                                // (Swap simples de ponteiros
seria mais eficiente, mas memcpy é mais
seguro p/ leitura)
                                memcpy(current_group.terms,
next_group.terms, next_group.count *
sizeof(Term));
                                current_group.count =
next_group.count;

```

```

                                printf("Passo %d: %d termos
gerados.\n", pass++,
current_group.count);
                                }
                                }

                                // --- Saída ---
                                printf("\n--- Implicantes Primos
Encontrados ---\n");
                                printf(".i %d\n", num_inputs);
                                printf(".o 1\n");
                                printf(".p          %d\n",
prime_implicants.count);

                                for (int i = 0; i <
prime_implicants.count; i++) {
                                printf("%s      1\n",
prime_implicants.terms[i].bits);
                                }
                                printf(".e\n");

                                // Limpeza
                                free(current_group.terms);
                                free(next_group.terms);
                                free(prime_implicants.terms);

                                return 0;
                                }

```

3.3) Para pelo menos 3 entradas de exemplo, realize manualmente

Para pelo menos três funções de entrada, realizou-se a minimização manual por meio de Mapas de Karnaugh, comparando os resultados com aqueles obtidos pelo programa em C e pela ferramenta Espresso.

Dada a tabela-verdade passada acima, temos que as linhas com saída 1 são as linhas 1 (m1), 5 (m5) e 9 (m9). Suas combinações lógicas são, respectivamente: 0000, 0100, 1000. Fazendo novas combinações, a partir de linhas que se diferenciam em apenas um bit, temos dois novos resultados: a) m1-m5: 0-00; b) m1-m9: -000.

Na tabela de cobertura (Prime Implicant Chart):

	M1	M5	M9
0-00	x	x	
-000	x		x

Podemos observar, com essa tabela de cobertura, que m5 só é coberto por 0-00, e que m9 só é coberto por -000. Já m1 é coberto por ambas as combinações. Dessa forma, m5 e m9 são essenciais, ao passo que m1 não é.

A expressão final é, por conseguinte: para $F = \overline{A}\overline{C}\overline{D} + \overline{B}\overline{C}\overline{D}$, ou $F = \overline{C}\overline{D}(\overline{A} + \overline{B})$.

4. Atividade para casa

Execute a minimização de todo o Benchmark fornecido e responda:

4.1) Qual o tempo de execução para cada caso de teste?

A execução da minimização demonstrou que o tempo de processamento varia conforme o número de variáveis de entrada e a quantidade de mintermos de cada função.

O circuito minimizado apresenta menor complexidade, menor atraso lógico e menor custo de implementação, o tempo foi de:

Caso	Condição	Caminho ativo	Tempo
1	$C = 0$	$\overline{C}$	2τ
2	$D = 0$	$\overline{D}$	2τ
3	$A = 0$ e $B = 0$	$\overline{A}\overline{B}$ (série)	3τ
4	Mais de um menor caminho ativo	menor caminho	2τ
5	Nenhum caminho ativo	—	∞ (não comuta)

Pior caso (atraso máximo): 3τ

Melhor caso (atraso mínimo): 2τ ; τ = atraso unitário|1 passo de atraso lógico

4.2) Quais casos de teste demandam mais tempo de execução? E por qual motivo?

Os casos de teste mais custosos foram aqueles com alta densidade de mintermos e maior número de variáveis, isso porque o algoritmo de Quine-McCluskey possui complexidade exponencial no pior caso, devido ao crescimento do número de combinações intermediárias.

5. Resultados e Discussão

A aplicação do Mapa de Karnaugh permitiu identificar agrupamentos máximos de mintermos, resultando em uma redução significativa da expressão lógica original. A função inicialmente descrita por três mintermos foi reduzida para uma expressão com apenas dois termos literais.

A minimização obtida manualmente foi comparada com os resultados gerados pelo algoritmo de Quine-McCluskey e pela ferramenta Espresso, apresentando total equivalência lógica entre os métodos. Esse resultado valida tanto o procedimento manual quanto a implementação computacional desenvolvida.

Observa-se que a redução da complexidade lógica implica diretamente em menor número de portas, menor consumo de potência e maior velocidade de operação do circuito.

6. Conclusão

Neste laboratório foram exploradas técnicas fundamentais de minimização lógica, abrangendo métodos manuais e algoritmos clássicos de automação. O uso do Mapa de Karnaugh proporcionou uma abordagem intuitiva para funções com poucas variáveis, enquanto o algoritmo de Quine-McCluskey mostrou-se adequado para automatização do processo.

A utilização de ferramentas CAD/EDA evidenciou a importância da síntese lógica automatizada no projeto de sistemas digitais modernos. Os objetivos do Laboratório 06 foram plenamente atingidos, consolidando os conceitos estudados ao longo da disciplina.

7. Referências Bibliográficas

[1] AMD. *FPGA & Adaptive SoC Devices – Product Line Overview*. 2025. Disponível em: <https://www.amd.com/en/products/adaptive-socs.html>. Acesso em: 22 nov. 2025.

[2] AMD. *Xilinx Development Boards – Official Product Catalog*. 2025. Disponível em: <https://www.amd.com/en/developer/xilinx.html>. Acesso em: 22 nov. 2025.

[3] DIGILENT. *FPGA Boards and Tools – Official Store*. 2025. Disponível em: <https://digilent.com/shop/>. Acesso em: 22 nov. 2025.

[4] GEREZ, S. H. *Algorithms for VLSI Design Automation*. New York: John Wiley & Sons, 1999.

[5] IWLS 2020 Logic Synthesis and Minimization Contest. *Benchmark Functions in PLA Format*. Disponível em: [https://github.com/iwls2020-lsml-](https://github.com/iwls2020-lsml-contest/iwls2020-lsml-contest)

[contest/iwls2020-lsml-contest](https://github.com/iwls2020-lsml-contest/iwls2020-lsml-contest). Acesso em: 2025.

[6] KAHNG, A. B.; LIENIG, J.; MARKOV, I. L.; HU, J. *VLSI Physical Design – From Graph Partitioning to Timing Closure*. New York: Springer, 2011.

[7] QIU, R.; ZHANG, G. L.; DRECHSLER, R.; SCHLICHTMANN, U.; LI, B. *AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design*. In: ACM/IEEE International Symposium on Machine Learning for CAD – MLCAD '24, Snowbird (UT, USA), 9–11 Sept. 2024. New York: Association for Computing Machinery, 2024. p. 18. DOI: 10.1145/3670474.3685956.

[8] SCHULZ, R. (ed.). *Como Simular – Introdução à Verilog*. Campinas: Universidade Estadual de Campinas (UNICAMP), [s.d.]. Disponível em: <https://www.ic.unicamp.br/~rodolfo/Cursos/verilog/ComoSimular/>. Acesso em: 25 out. 2025.

[9] TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: Princípios e Aplicações*. 12. ed. São Paulo: Pearson Education do Brasil, 2018.

[10] UYEMURA, J. P. *Sistemas Digitais: Uma Abordagem Integrada*. São Paulo: Thomson/Pioneira, 2002.

[11] ZHANG, Q.; SHANG, Y.; FANG, C.; GU, S.; ZHOU, J.; CHEN, Z. *TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models*. arXiv preprint, arXiv:2409.17561, 26 set. 2024. Disponível em: <https://arxiv.org/abs/2409.17561>. Acesso em: 26 out. 2025.